

STRUCTURATION DE DOCUMENTS

Membres du groupe

ABOTSI Kwassi Nyuieva Junior

Nathan PIERROT

I.	Introduction	3
II.	Conception	3
A.	Choix général de MongoDB	3
B.	Organisation de la base de données	4
C.	Collection G_AP_sources	4
D.	Collection G_AP_articles	5
E.	Collection G_AP_consultations	6
F.	Relations logiques entre les collections	6
G.	Patrons de structuration utilisés	7
H.	Indexation	7
III.	Développement	8
A.	Organisation des fichiers	8
B.	Lecture des sitemaps XML	9
C.	Gestion des sources et des minuteriers	9
D.	Insertion et mise à jour des articles	9
E.	Mode consultation	9
F.	Historique des consultations	10
G.	Génération du nuage de mots SVG	10
H.	Interface Flask	11
I.	Gestion des dates et fuseaux horaires	12
J.	Organisation du travail avec GitHub	12
IV.	Bilan des fonctionnalités	12
V.	Difficultés rencontrées	13
VI.	Conclusion	14

I. Introduction

Ce projet a été réalisé dans le cadre de l'unité d'enseignement Structuration de documents. Le sujet porte sur la collecte, l'organisation et la visualisation de titres d'actualité à partir de sources web fiables. L'objectif principal est de développer une application web qui permet de s'abonner à plusieurs sources d'actualités via leurs sitemaps, de récupérer les titres publiés, de les stocker dans une base MongoDB, puis de les exploiter dans une interface de consultation et sous forme de nuage de mots au format SVG.

Le projet reprend plusieurs notions vues en cours et en TD. La partie XML intervient lors de la lecture des sitemaps, qui sont des documents structurés sous forme d'arbre. La partie JSON/BSON intervient avec MongoDB, car les sources, les articles et les consultations sont stockés sous forme de documents. Les requêtes, l'indexation et certaines opérations de mise à jour s'appuient directement sur les TD MongoDB.

L'application fonctionne en deux modes. Le mode administration sert à gérer les abonnements aux sitemaps : ajout d'une source, suppression d'un abonnement, définition d'une fréquence de récupération et récupération manuelle ou programmée. Le mode consultation permet de parcourir les articles récupérés, de filtrer les résultats par source, date de publication, texte ou mot-clé, puis d'ouvrir l'article original. Lorsqu'un titre est ouvert, l'action est horodatée afin de garder une trace des consultations.

Enfin, l'application génère un nuage de mots à partir des titres collectés. L'utilisateur choisit une période et le nombre de mots à afficher. Le résultat est produit en SVG, ce qui permet à la fois de l'afficher dans le navigateur et de le télécharger.

II. Conception

A. Choix général de MongoDB

MongoDB est une bonne solution de stockage de données parce que le projet manipule des données semi-structurées. MongoDB est une base NoSQL orientée documents : une base contient des collections, et une collection contient des documents représentés sous une forme proche du JSON.

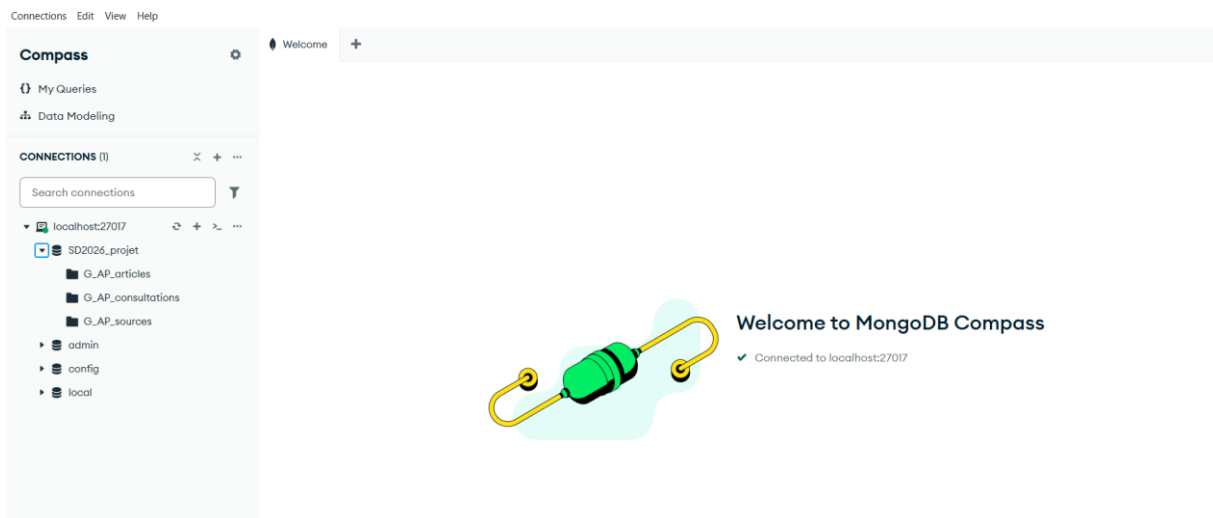
Ce modèle convient bien au projet. Une source d'actualité possède un nom, une URL de sitemap, une fréquence de récupération et une date de dernière récupération. Un article possède un titre, une URL, une date de publication, éventuellement des mots-clés et parfois une image. Une consultation possède une date et une référence vers l'article ouvert. Toutes ces informations peuvent être représentées naturellement sous forme de documents. Et s'il manque des informations, la structure s'adapte automatiquement.

Utiliser MongoDB nous permet de mettre réellement en application les notions vues en cours et dans les TDs : insertion de documents, recherche avec find, mise à jour avec update, suppression avec delete, indexation et analyse des requêtes. Avec le projet, on ne veut pas utiliser un modèle relationnel, mais plutôt une structuration plus adaptée aux documents.

B. Organisation de la base de données

Comme demandé dans le sujet, nous avons appelé notre base SD2026_projet. Et nos collections sont, pour notre groupe ABOTSI / PIERROT, préfixées de G_AP_. Voici une vue globale des différentes collections de notre projet :

Collection	Rôle dans l'application
G_AP_sources	Stocke les abonnements aux sitemaps des journaux ou sites d'actualité.
G_AP_articles	Stocke les articles récupérés depuis les sitemaps.
G_AP_consultations	Stocke l'historique des clics sur les articles.



C. Collection G_AP_sources

Cette collection représente les sources d'actualité auxquelles l'utilisateur s'abonne. Chaque document correspond à un sitemap enregistré dans l'application.

Champ	Description
_id	Identifiant MongoDB de la source.
nom	Nom affiché dans l'interface.
sitemap_url	URL du sitemap utilisé pour récupérer les articles.

active	Indique si la source est active.
frequence_heures	Minuterie de récupération exprimée en heures.
date_abonnement	Date d'ajout de la source.
derniere_recuperation	Date de la dernière récupération effectuée.

```
{
  "nom": "Le Monde",
  "sitemap_url": "https://www.lemonde.fr/sitemap_news.xml",
  "active": true,
  "frequence_heures": 24,
  "date_abonnement": "...",
  "derniere_recuperation": "..."
}
```

D. Collection G_AP_articles

Cette collection contient les titres d'actualité récupérés depuis les sitemaps. Chaque document correspond à un article. Le champ `source_id` relie l'article à sa source, tandis que `source_nom` est conservé aussi dans le document pour simplifier l'affichage par journal dans l'interface.

Champ	Description
_id	Identifiant MongoDB de l'article.
source_id	Identifiant de la source qui a fourni l'article.
source_nom	Nom de la source recopié pour simplifier l'affichage.
titre	Titre de l'article.
url	Lien vers l'article original.
date_publication	Date de publication utilisée dans les filtres et le nuage.
mots_cles	Liste de mots-clés récupérés si le sitemap les fournit.
image_url	Image associée si elle est disponible.
date_recuperation	Date à laquelle l'article a été récupéré.

```
{
  "source_id": ObjectId("..."),
```

```
"source_nom": "Le Monde",
"titre": "Exemple de titre d'actualité",
"url": "https://...",
"date_publication": "...",
"mots_cles": ["politique", "économie"],
"image_url": "https://...",
"date_recuperation": ".."
}
```

E. Collection G_AP_consultations

Cette collection garde l'historique des articles réellement ouverts par l'utilisateur. Lorsqu'un utilisateur clique sur un titre, l'application enregistre une consultation avant de rediriger vers l'article original.

Champ	Description
_id	Identifiant MongoDB de la consultation.
article_id	Identifiant de l'article consulté.
source_id	Identifiant de la source de l'article.
titre	Titre recopié pour faciliter l'affichage de l'historique.
url	URL de l'article original.
date_consultation	Date et heure du clic.

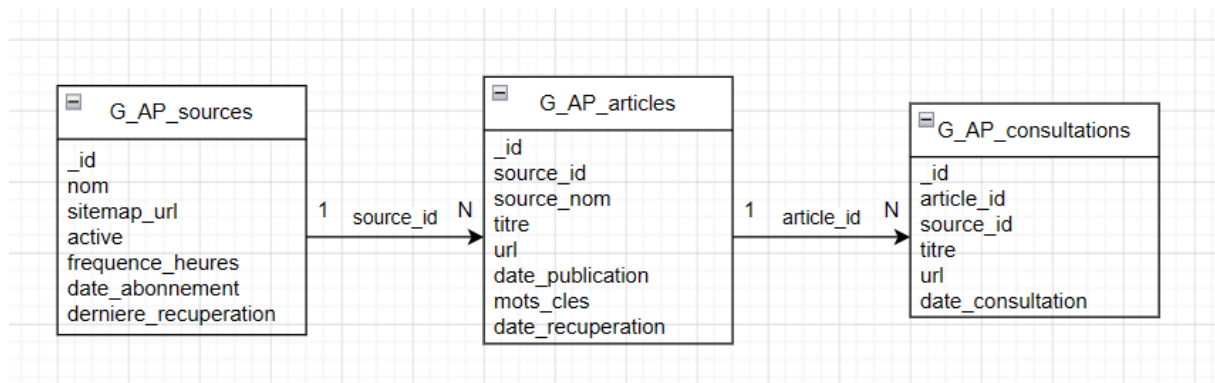
F. Relations logiques entre les collections

Même si MongoDB ne fonctionne pas comme une base relationnelle classique, les documents restent liés grâce à des id. Ces liens sont gérés par l'application :

G_AP_sources 1 — N G_AP_articles
lien : source_id

G_AP_articles 1 — N G_AP_consultations
lien : article_id

Ce choix évite d'utiliser des jointures lourdes pour l'affichage courant. Ça reste cohérent avec les principes NoSQL, les relations ne sont pas forcément explicites dans la base et une partie de la logique est portée par l'application cliente.



G. Patrons de structuration utilisés

Le projet s'appuie surtout sur deux idées vues dans la partie du cours consacrée à la structuration de documents NoSQL.

Patron	Utilisation dans le projet
Référence étendue	Les articles gardent source_id mais aussi source_nom. Cela évite une recherche supplémentaire pour afficher les titres avec leur source.
Sous-éléments	Les mots-clés d'un article sont stockés dans un tableau mots_cles, directement dans le document article.
Souplesse du document	Certains champs comme image_url ou mots_cles peuvent être vides selon la structure du sitemap récupéré.

Le projet ne force donc pas tous les articles à avoir exactement les mêmes informations. Cela correspond à la logique de MongoDB : les documents d'une même collection ont souvent une structure similaire, mais pas forcément strictement identique.

H. Indexation

Les index ont été choisis en fonction des recherches réellement utilisées dans l'application. Comme vu en TD, ils permettent d'éviter dans certains cas un parcours complet de la collection. En contrepartie, ils ont un coût en stockage, il ne fallait donc pas indexer tous les champs, seulement les champs les plus utilisés.

Index	Justification
G_AP_sources.sitemap_url unique	Évite d'ajouter deux fois le même sitemap.
G_AP_sources.active	Facilite la récupération des sources actives.
G_AP_articles.url unique	Évite les doublons lors des récupérations successives.
G_AP_articles.source_id + date_publication	Utile pour afficher les articles d'une source sur une période.
G_AP_articles.date_publication	Utile pour filtrer les articles et construire le nuage.
G_AP_articles.mots_cles	Utile pour la recherche par mot-clé.
G_AP_consultations.date_consultation	Utile pour filtrer l'historique par date et heure.
G_AP_consultations.article_id	Utile pour retrouver les consultations liées à un article.

III. Développement

A. Organisation des fichiers

Le projet est organisé autour d'un dossier src. Chaque fichier a une fonction précise, ce qui permet de garder un code lisible et d'avoir une structure simple pour le projet.

Fichier et dossiers	Rôle
src/main.py	Point d'entrée Flask et définition des routes HTTP.
src/BdMongo.py	Connexion MongoDB, création des index, opérations d'ajout, modification, suppression.
src/config.py	Paramètres de configuration : hôte, port, nom de base et noms des collections.
src/sitemap.py	Lecture des sitemaps XML et extraction des articles.

<code>src/articles.py</code>	Gestion de la récupération selon les fréquences configurées.
<code>src/wordcloud_svg.py</code>	Calcul des mots fréquents et génération du SVG.
<code>src/templates/</code>	Pages HTML.
<code>src/static/css/</code>	Feuilles de style CSS.

B. Lecture des sitemaps XML

Pour récupérer des articles on commence par lire les sitemaps, organisés en nœuds. Et on recherche principalement les informations suivantes : loc, title, publication_date, lastmod et keywords.

Dans notre code, on prend aussi en compte les espaces de noms XML. Et pour éviter que les namespaces rendent le parsing trop fragile, on a écrit une fonction qui récupère seulement le nom local de la balise. Cela permet de lire les balises utiles même lorsque le sitemap utilise des préfixes différents.

On essaye aussi de récupérer les sitemaps index, c'est des sitemaps qui renvoient vers d'autres sitemaps. Comme notre projet concerne l'actualité, on priorise ceux qui contiennent mot news.

C. Gestion des sources et des minuteriers

Dans mode administration on peut ajouter une source avec un nom, une URL de sitemap et une fréquence exprimée en heures. La fréquence permet de savoir si une source doit être récupérée à nouveau. Pour cela on compare la date actuelle avec le champ `derniere_recuperation` et la valeur `frequence_heures`.

La récupération peut aussi être lancée manuellement depuis l'interface. Après une récupération, la date de dernière récupération de la source est mise à jour dans MongoDB pour pouvoir relancer la récupération une fois le minuteur écoulé.

D. Insertion et mise à jour des articles

Quand un article est récupéré, il est inséré ou mis à jour dans `G_AP_articles`. La méthode utilisée est un upsert sur l'URL de l'article. Nous avons choisi cette méthode parce qu'un même sitemap peut être relu plusieurs fois : l'index unique sur url et l'upsert empêchent d'avoir des soucis d'unicité.

E. Mode consultation

Le mode consultation nous permet de parcourir les articles récupérés. On peut y voir une colonne par source. On a décidé d'ajouter quelques filtres pour que l'utilisateur

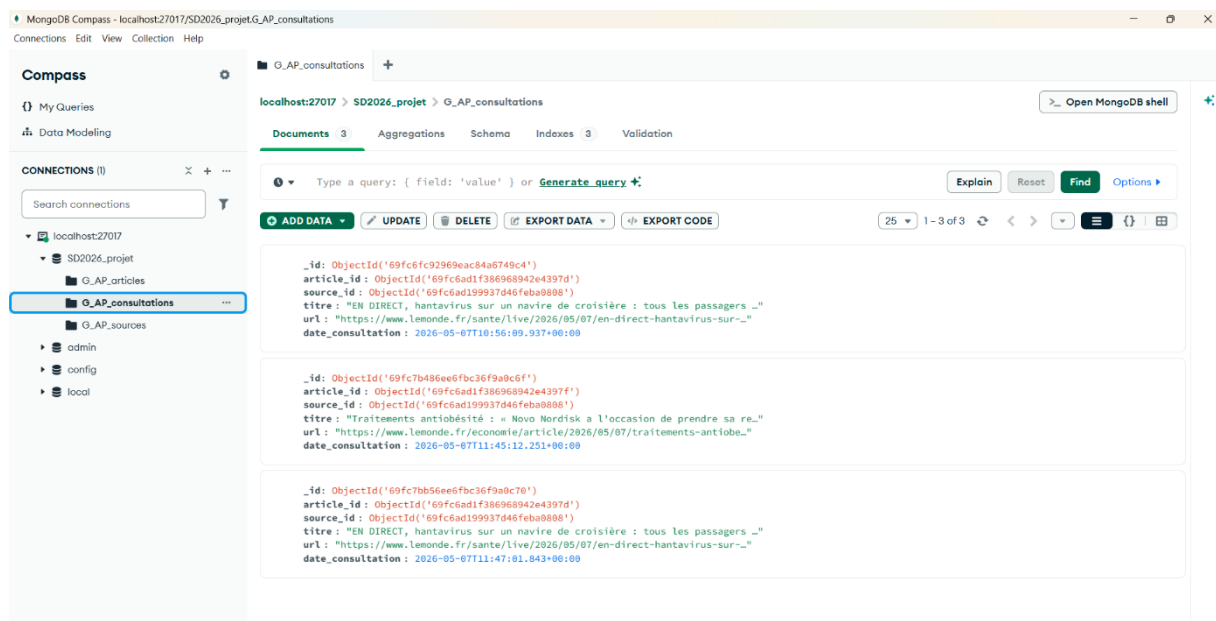
puisse plus facilement retrouver les articles qui l'intéressent : source, date de début, date de fin, texte dans le titre et mot-clé.

Une fois que le bouton de recherche est appuyé, on construit une requête selon les champs remplis par l'utilisateur. Par exemple, une période crée une condition sur `date_publication` avec `$gte` et `$lte`. Une recherche dans le titre utilise une expression régulière. La recherche par mot-clé s'applique sur le champ `mots_cles`. Exemple :

```
requete["date_publication"] = {"$gte": date_debut, "$lte": date_fin}
requete["titre"] = {"$regex": texte, "$options": "i"}
```

F. Historique des consultations

Quand un utilisateur clique sur un titre, on récupère l'url cliqué puis on enregistre une consultation dans `G_AP_consultations`, et enfin redirige l'utilisateur vers l'url cliquée. Grâce à cela on peut garder mémoire l'heure de consultation pour par la suite pouvoir retrouver les articles déjà consultés.



Cette capture d'écran montre la structure des consultations de l'utilisateur dans la collection `G_AP_consultations`. Chaque consultation enregistre l'identifiant de l'article, la source, le titre, l'url et la date et l'heure de consultation.

G. Génération du nuage de mots SVG

On construit le nuage de mots à partir des titres des articles publiés dans la période choisie. L'utilisateur peut choisir le nombre de mots à afficher.

Étape	Description
1	Sélectionner les titres dans la période demandée.
2	Découper les titres en mots.
3	Ignorer les mots très fréquents mais peu informatifs : le, la, de, du, des, etc.
4	Compter les fréquences avec un compteur.
5	Garder les n mots les plus fréquents.
6	Générer un SVG affichable et téléchargeable.

Dans le SVG, plus un mot est fréquent plus il va être affiché en gros et au centre. Pour placer les mots, on le fait de manière progressive à partir du centre et on applique une vérification de collisions pour éviter que les mots se superposent trop fortement. Il peut y avoir de légères superpositions mais elles n'empêchent pas la lecture. Nous avons décidé de les laisser pour obtenir un nuage de mot bien compact sans trop d'espace entre les mots.

H. Interface Flask

Route	Méthode	Rôle
/admin	GET	Afficher la gestion des sources.
/admin/sources	POST	Ajouter une source.
/admin/sources/<id>/supprimer	POST	Supprimer une source.
/admin/sources/<id>/recuperer	POST	Récupérer manuellement une source.
/admin/sources/recuperer- dues	POST	Récupérer les sources dont la minuterie est échue.
/consultation	GET	Afficher les articles avec filtres.
/articles/<id>/ouvrir	GET	Enregistrer le clic puis ouvrir l'article original.
/historique	GET	Afficher les consultations enregistrées.

/nuage	GET	Afficher le formulaire et le nuage SVG.
/nuage/telecharger	GET	Télécharger le nuage au format SVG.

I. Gestion des dates et fuseaux horaires

Pendant nos tests nous avons constaté que nous avions un décalage de 2 heures sur les horaires de consultations des articles. Après avoir effectué quelques recherches, on s'est rendu compte que MongoDB stockait les dates en UTC. Donc pour éviter les décalages dans l'affichage, on convertit les dates automatiquement vers notre fuseau horaire Europe/Paris avant de les afficher.

J. Organisation du travail avec GitHub

Nous avons décidé d'utiliser GitHub afin de garder un historique des versions et de simplifier le travail en binôme. Voici l'url du dépôt distant :

<https://github.com/Ratz123323/Structuration-de-document>

Utiliser de Git nous a permis de garder un historique des différentes étapes du développement de l'application. On a ajouté les principales fonctionnalités une à une : mise en place de l'application Flask, connexion à MongoDB, gestion des abonnements aux sitemaps, récupération des articles, consultation des données, historique des consultations et génération du nuage de mots SVG.

En développant de manière progressive, on a pu tester progressivement le code qu'on ajoutait au projet. Cela nous a fait gagner du temps car nous avons pu voir les bugs présents sans avoir trop développé par-dessus.

GitHub nous a également permis de centraliser tout le code du projet et de conserver une structure de travail propre.

IV. Bilan des fonctionnalités

Afin de voir globalement toutes les fonctionnalités que nous avons développé dans ce projet, nous les avons résumés dans ce tableau :

Partie	Fonctionnalité
Administration	Ajout d'un abonnement à un sitemap
Administration	Suppression d'un abonnement
Administration	Définition d'une fréquence de récupération
Administration	Récupération manuelle d'une source
Administration	Récupération des sources dues selon minuterie
Consultation	Consultation globale des articles
Consultation	Consultation par source
Consultation	Filtre par date de publication
Consultation	Recherche par texte dans le titre
Consultation	Recherche par mot-clé
Consultation	Ouverture de l'article original
Historique	Horodatage du clic
Historique	Recherche par date et heure de consultation
Nuage de mots	Choix d'une période
Nuage de mots	Choix du nombre de mots affiché
Nuage de mots	Affichage SVG
Nuage de mots	Téléchargement SVG
Base de données	Base SD2026_projet
Base de données	Collections avec préfixe G_AP_
Base de données	Indexation des champs utiles

V. Difficultés rencontrées

Lors de la réalisation de ce projet, nous avons rencontré quelques difficultés, que nous détaillons dans cette partie :

- Hétérogénéité des sitemaps : Tous les sites ne structurent pas leurs sitemaps de la même manière, certains utilisent des namespaces différents, certains ont des sitemaps index, et certains ne fournissent pas toutes les informations attendues.

Une fois qu'on a compris ça, on a rendu la récupération plus globale et adaptative en fonction du type de données du site.

- Informations parfois absentes : Certains sitemaps ne fournissent pas de titre, d'image ou de mots-clés. On a décidé, puisque la structure de MongoDB le permet, de garder les champs vides ou de reconstruire un titre lisible à partir de l'URL.
- Doublons possibles : Une même source peut être récupérée plusieurs fois donc pour éviter les doublons, on a décidé d'indexer l'URL de l'article de façon unique et de faire l'insertion avec un upsert.
- Gestion des dates : Comme les dates sont stockées en UTC dans MongoDB, ça créait des décalages horaires dans l'affichage. On a donc converti l'heure au fuseau Europe/Paris.
- Placement dans le nuage SVG : Au départ, on avait beaucoup de superpositions entre les mots. On a donc décidé de placer une « boîte » sur chaque mot et de vérifier les superpositions de ces boîtes pour limiter les superpositions des mots.

VI. Conclusion

Ce projet a permis de mettre en pratique plusieurs notions cours. Notamment la lecture et l'utilisation des sitemaps et donc des documents XML et le stockage des données dans une base NoSQL avec MongoDB.

MongoDB s'adaptait parfaitement au projet parce que les données d'actualité ne sont pas toujours strictement identiques d'une source à l'autre. Certains articles possèdent une image, d'autres non ; certains sitemaps fournissent des mots-clés, d'autres non. Et une base de données relationnelle aurait été trop rigide pour stocker correctement ces données.

L'utilisation d'index nous a permis d'optimiser les recherches les plus fréquentes : recherche par source, par date de publication, par mot-clé et par date de consultation. Ça évite de parcourir toute la collection document par document et d'avoir des temps de réponse trop élevés.

On a réussi à développer les fonctionnalités demandées par le sujet mais quelques améliorations restent possibles. Par exemple, la liste des mots ignorés pourrait être automatique en fonction de la taille des mots. On pourrait aussi améliorer le rendu du nuage de mots, ou afficher plusieurs propositions pour laisser l'utilisateur choisir selon ses préférences. Ces pistes restent ouvertes pour un approfondissement du sujet.